PROBLEMI DI SICUREZZA COMUNI NELLE WEBAPP

Leonardo Querzoni
querzoni@dis.uniroma1.it

SAPIENZA
UNIVERSITÀ DI ROMA

CIS SAPIENZA
CYBER INTELLIGENCE AND INFORMATION SECURITY

DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI

PREMESSA

Molte delle seguenti slide provengono dal sito web OWASP

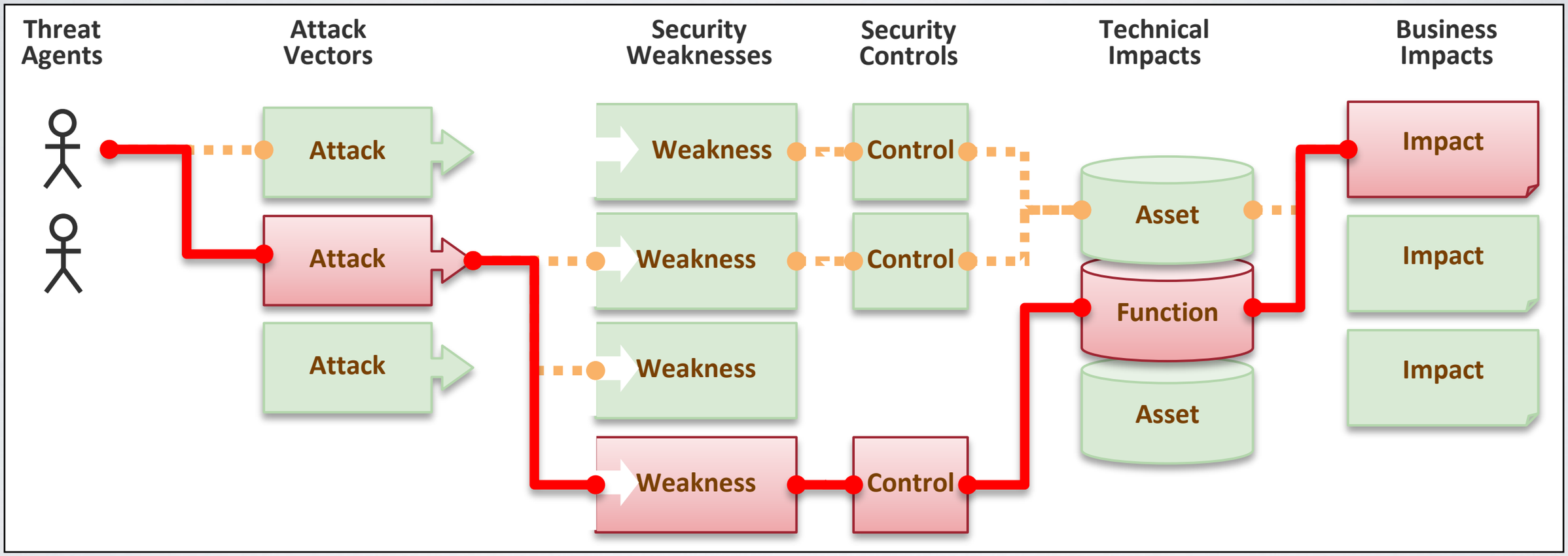
- OWASP Top-10 project
  https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project

OWASP TOP10 PROJECT

- OWASP Top 10 è un documento orientato alla consapevolezza
  - Non è uno standard
- Prima edizione nel 2003
  - Poi: 2004, 2007, 2010, 2013
- Fornisce schede riassuntive sui 10 rischi più comuni a cui sono esposte le applicazioni web. Per ciascuna dettaglia:
  - Il livello di rischio prevedibile
  - Come il rischio può essere giustificato dalla presenza di una vulnerabilità
  - Quali contromisure adottare per eliminare o contenere il rischio

OWASP TOP10 PROJECT

In che modo una vulnerabilità può diventare un attacco?



OWASP TOP10 PROJECT

Per ogni rischio vengono definiti attraverso una semplice scala

- La semplicità di sfruttamento da parte di un attaccante
- La prevalenza globale
- La semplicità di identificazione dell'attacco
- L'impatto

Threat Agents	Attack Vectors	Weakness Prevalence	Weakness Detectability	Technical Impacts	Business Impacts
App Specific	Easy	Widespread	Easy	Severe	App / Business Specific
	Average	Common	Average	Moderate	
	Difficult	Uncommon	Difficult	Minor	

OWASP TOP10 PROJECT

A1 Injection	A2 Broken Authentication and Session Management	A3 Cross-Site Scripting (XSS)	A4 Insecure Direct Object References	A5 Security Misconfiguration
A6 Sensitive Data Exposure	A7 Missing Function Level Access Control	A8 Cross Site Request Forgery (CSRF)	A9 Using Known Vulnerable Components	A10 Unvalidated Redirects and Forwards

A1 – INJECTION

Injection means…

- Tricking an application into including unintended commands in the data sent to an interpreter

Interpreters…

- Take strings and interpret them as commands
- SQL, OS Shell, LDAP, XPath, Hibernate, etc

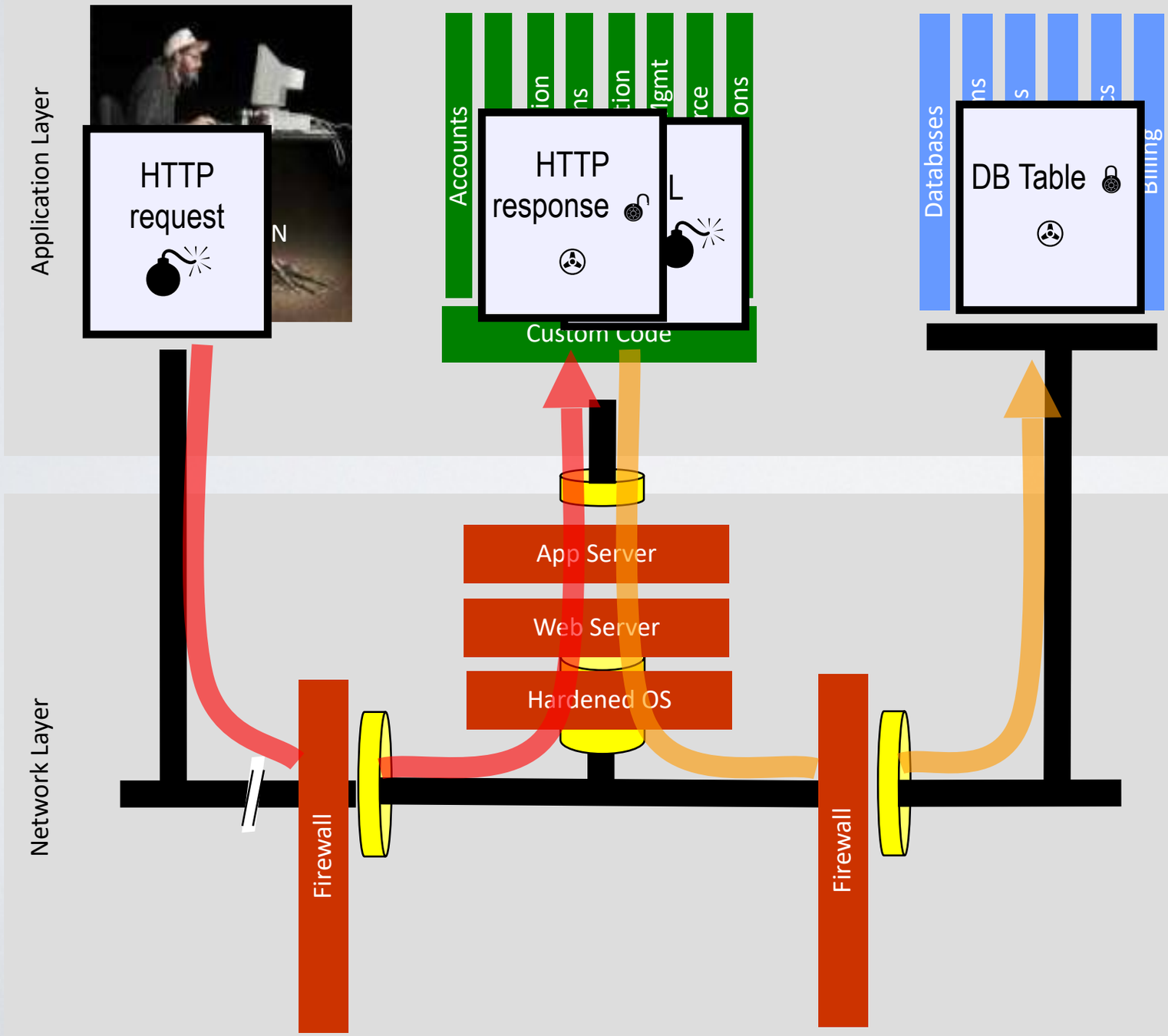
SQL injection is still quite common

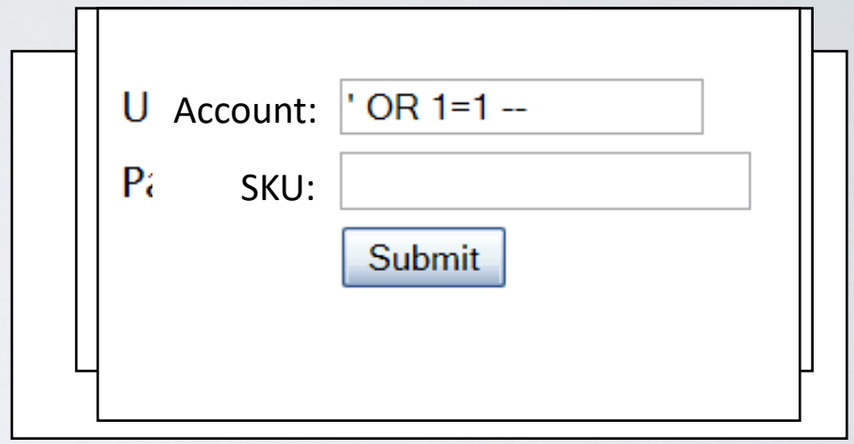
- Many applications still susceptible (really don't know why)
- Even though it's usually very simple to avoid

Typical Impact

- Usually severe. Entire database can usually be read or modified
- May also allow full database schema, or account access, or even OS level access

SQL INJECTION – ILLUSTRATED





1. Application presents a form to the attacker

2. Attacker sends an attack in the form data

3. Application forwards attack to the database in a SQL query

4. Database runs query containing attack and sends encrypted results back to application

5. Application decrypts data as normal and sends results to the user

A1 – AVOIDING INJECTION FLAWS

Recommendations

- Avoid the interpreter entirely, or
- Use an interface that supports bind variables (e.g., prepared statements, or stored procedures),
  - Bind variables allow the interpreter to distinguish between code and data
- Encode all user input before passing it to the interpreter
- Always perform ‘white list’ input validation on all user supplied input
- Always minimize database privileges to reduce the impact of a flaw

For more details: https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet

A2 – BROKEN AUTH AND SESSION MANAGEMENT

HTTP is a “stateless” protocol

- Means credentials have to go with every request
- Should use SSL for everything requiring authentication

Session management flaws

- SESSION ID used to track state since HTTP doesn’t
  - and it is just as good as credentials to an attacker
- SESSION ID is typically exposed on the network, in browser, in logs, …

Beware the side-doors

- Change my password, remember my password, forgot my password, secret question, logout, email address, etc…

Typical Impact

- User accounts compromised or user sessions hijacked

BROKEN AUTHENTICATION

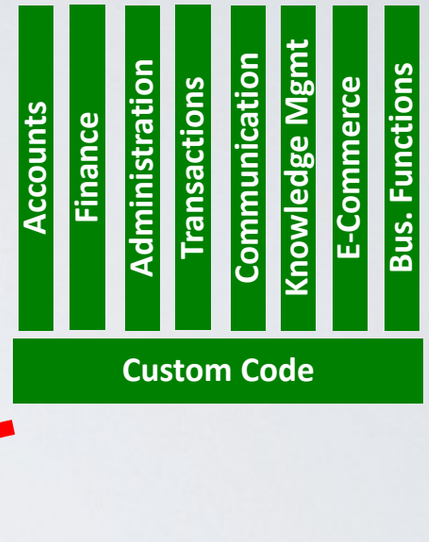
1 **User sends credentials**

www.boi.com?JSESSIONID=9FA1DB9EA...

Site uses URL rewriting

(i.e., put session in URL)

2

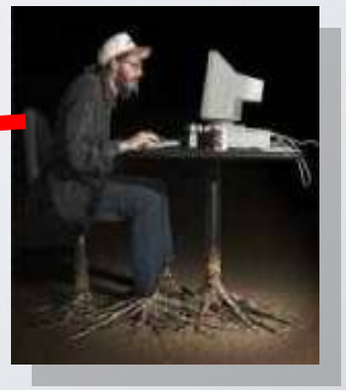


3 **User clicks on a link to http://www.hacker.com in a forum**

Hacker checks referrer logs on www.hacker.com

and finds user's JSESSIONID

4



5 **Hacker uses JSESSIONID and takes over victim's account**

A2 – BROKEN AUTH AND SESSION MANAGEMENT

Verify your architecture

- Authentication should be simple, centralized, and standardized
- Use the standard session id provided by your container
- Be sure SSL protects both credentials and session id at all times

Verify the implementation

- Forget automated analysis approaches
- Check your SSL certificate
- Examine all the authentication-related functions
- Verify that logoff actually destroys the session
- Use OWASP's WebScarab to test the implementation

Follow the guidance from

- https://www.owasp.org/index.php/Authentication_Cheat_Sheet

A3 – CROSS-SITE SCRIPTING

Occurs any time…

- Raw data from attacker is sent to an innocent user's browser

Raw data…

- Stored in database (Persistent XSS)
- Reflected from client (Reflected XSS)

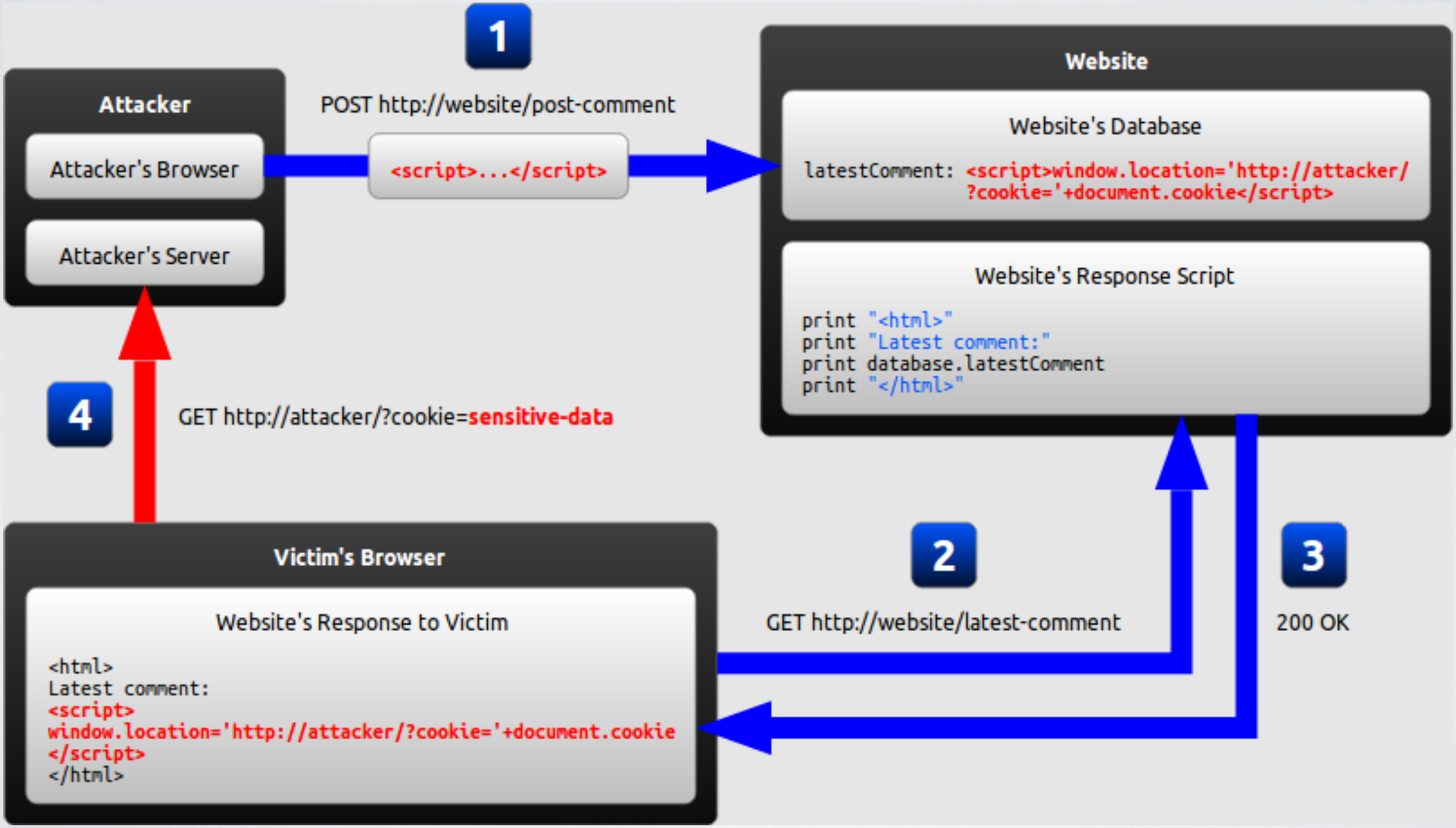
A lot of web applications have this problem

Typical Impact

- Steal user's session, steal sensitive data, rewrite web page, redirect user to phishing or malware site
- Most Severe: Install XSS proxy which allows attacker to observe and direct all user's behavior on vulnerable site and force user to other sites

CROSS-SITE SCRIPTING

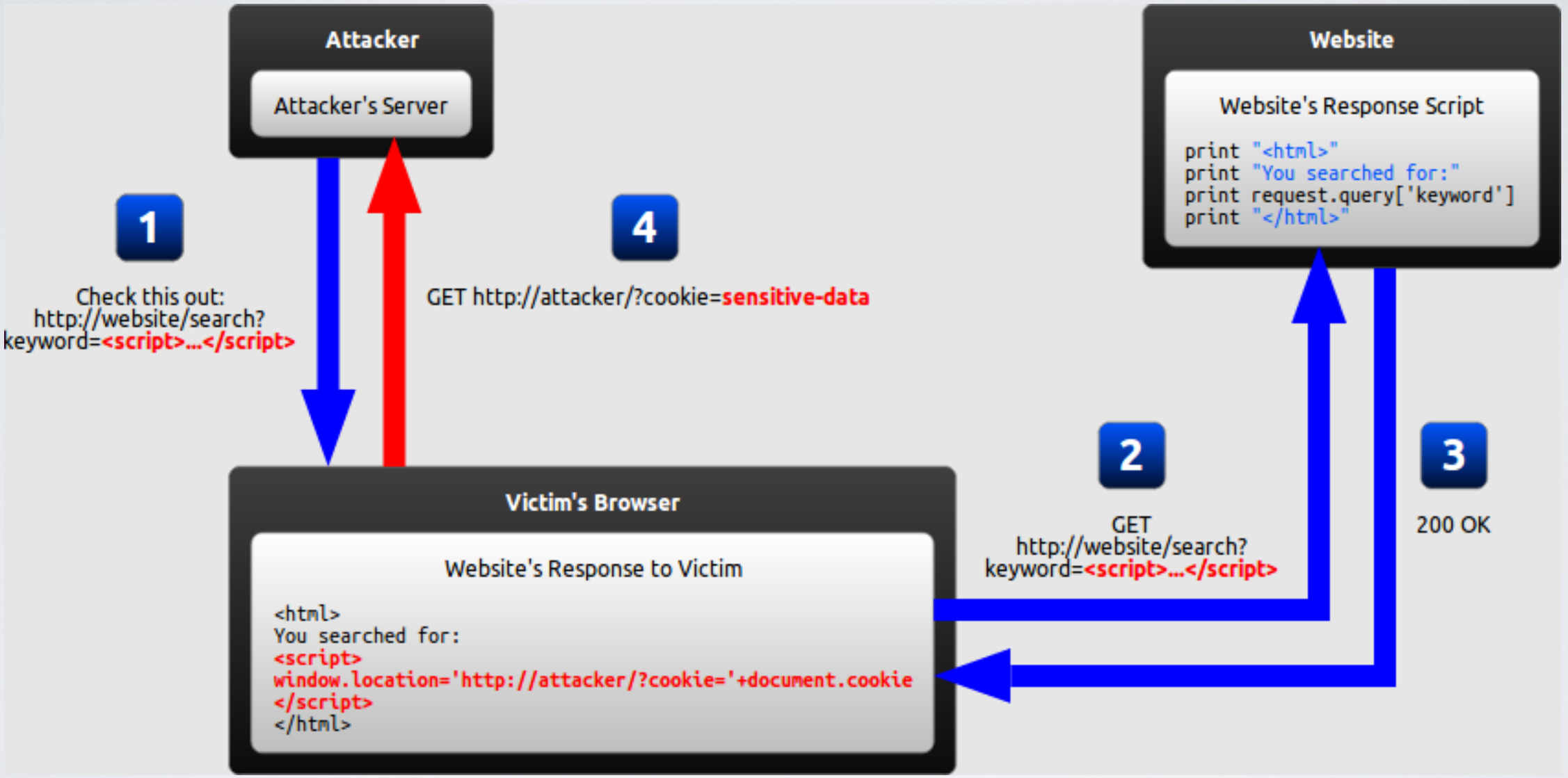
- Example: persistent XSS



source: excess-xss.com

CROSS-SITE SCRIPTING

- Example: reflected XSS



source: excess-xss.com

AVOIDING XSS FLAWS

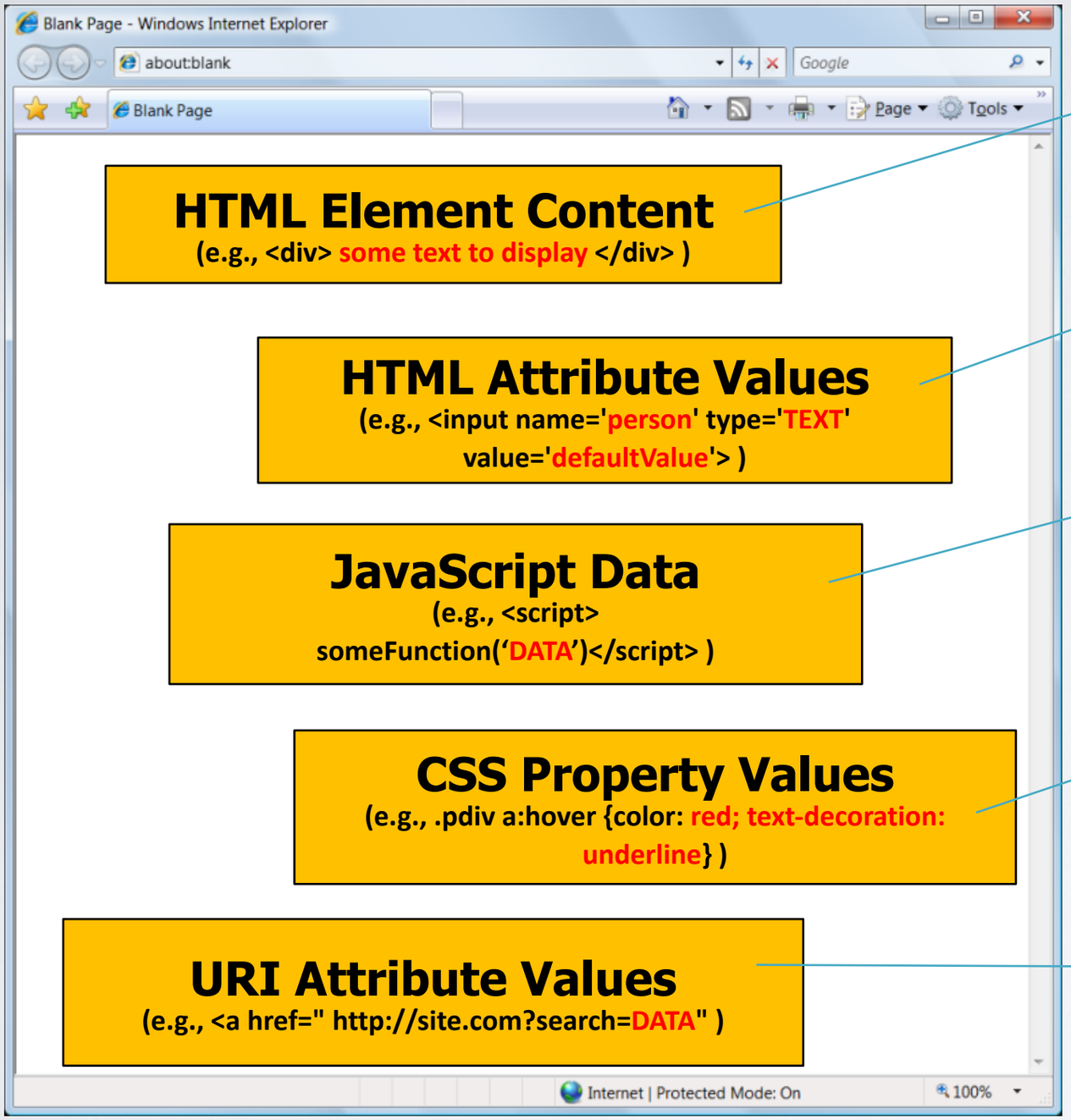
Recommendations

- Eliminate Flaw
  - Don’t include user supplied input in the output page
- Defend Against the Flaw
  - Use Content Security Policy (CSP)
  - Primary Recommendation: Output encode all user supplied input (Use OWASP’s ESAPI or Java Encoders to output encode)
  - Perform ‘white list’ input validation on all user input to be included in page
  - For large chunks of user supplied HTML, use OWASP’s AntiSamy to sanitize this HTML to make it safe

References

- https://www.owasp.org/index.php/XSS_(Cross Site Scripting) Prevention Cheat Sheet

SAFE ESCAPING SCHEMES IN VARIOUS HTML EXECUTION CONTEXTS



#1: (&, <, >, ") → &entity; (', /) → &#xHH;
ESAPI: encodeForHTML()

#2: All non-alphanumeric < 256 → &#xHH;
ESAPI: encodeForHTMLAttribute()

#3: All non-alphanumeric < 256 → \xHH
ESAPI: encodeForJavaScript()

#4: All non-alphanumeric < 256 → \HH
ESAPI: encodeForCSS()

#5: All non-alphanumeric < 256 → %HH
ESAPI: encodeForURL()

ALL other contexts CANNOT include Untrusted Data

Recommendation: Only allow #1 and #2 and disallow all others

See: www.owasp.org/index.php/XSS_(Cross_Site_Scripting)_Prevention_Cheat_Sheet

A4 – INSECURE DIRECT OBJECT REFERENCES

How do you protect access to your data?

- This is part of enforcing proper “Authorization”, along with A7 – Failure to Restrict URL Access

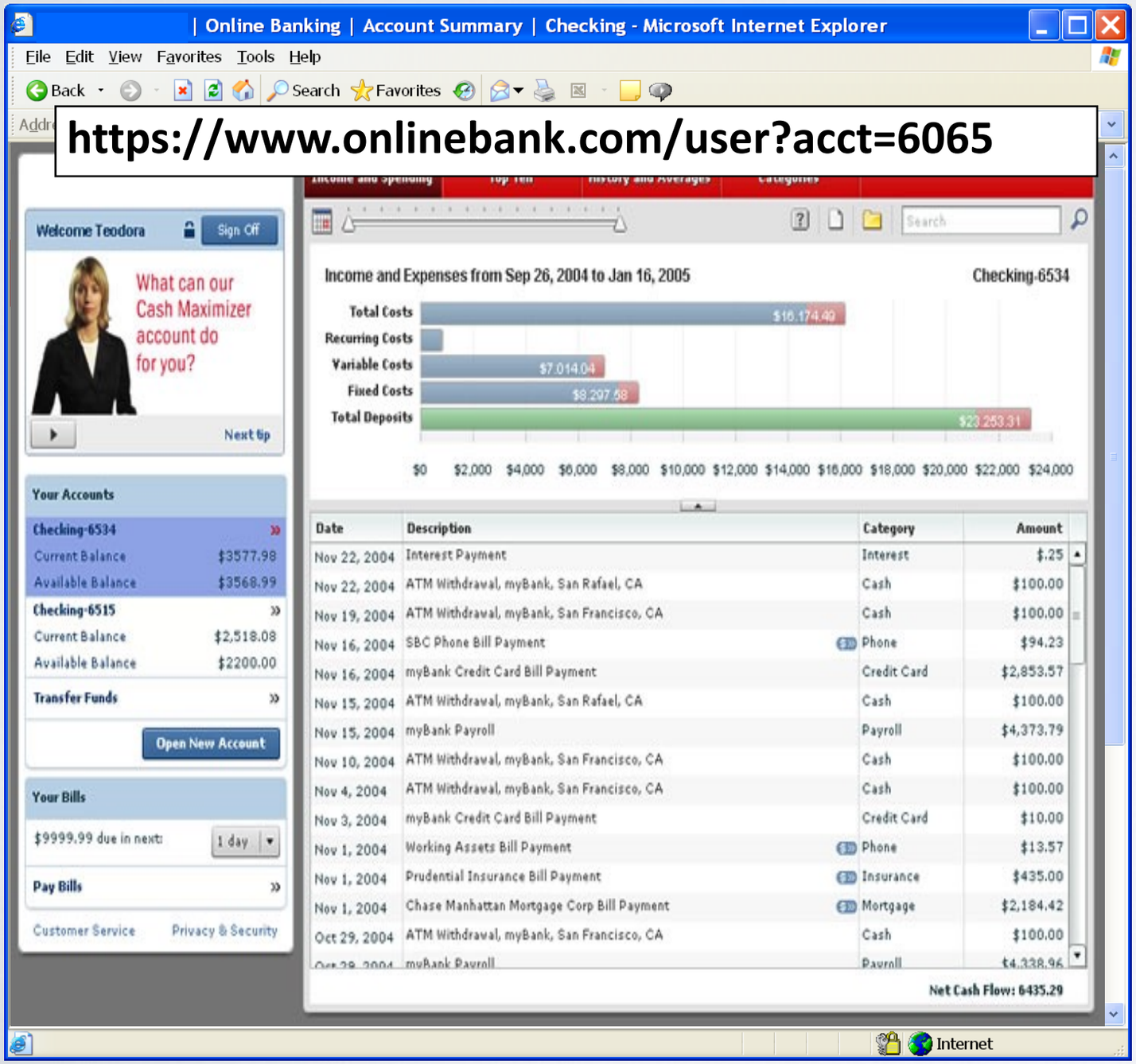
A common mistake …

- Only listing the ‘authorized’ objects for the current user, or
- Hiding the object references in hidden fields
- … and then not enforcing these restrictions on the server side
- This is called presentation layer access control, and doesn’t work
  - Attacker simply tampers with parameter value

Typical Impact

- Users are able to access unauthorized files or data

INSECURE DIRECT OBJECT REFERENCES



- Attacker notices his acct parameter is 6065

  ?acct=6065

- He modifies it to a nearby number

  ?acct=6066

- Attacker views the victim's account information

A4 – INSECURE DIRECT OBJECT REFERENCES

Eliminate the direct object reference

- Replace them with a temporary mapping value (e.g. 1, 2, 3)

http://app?file=Report123.xls
http://app?file=1

http://app?id=9182374
http://app?id=7d3J93

Report123.xls

Acct:9182374

Validate the direct object reference

- Verify the parameter value is properly formatted
- Verify the user is allowed to access the target object
  - Query constraints work great!
- Verify the requested mode of access is allowed to the target object (e.g., read, write, delete)

A5 – SECURITY MISCONFIGURATION

Web applications rely on a secure foundation

- Everywhere from the OS up through the App Server

Is your source code a secret?

- Think of all the places your source code goes
- Security should not require secret source code

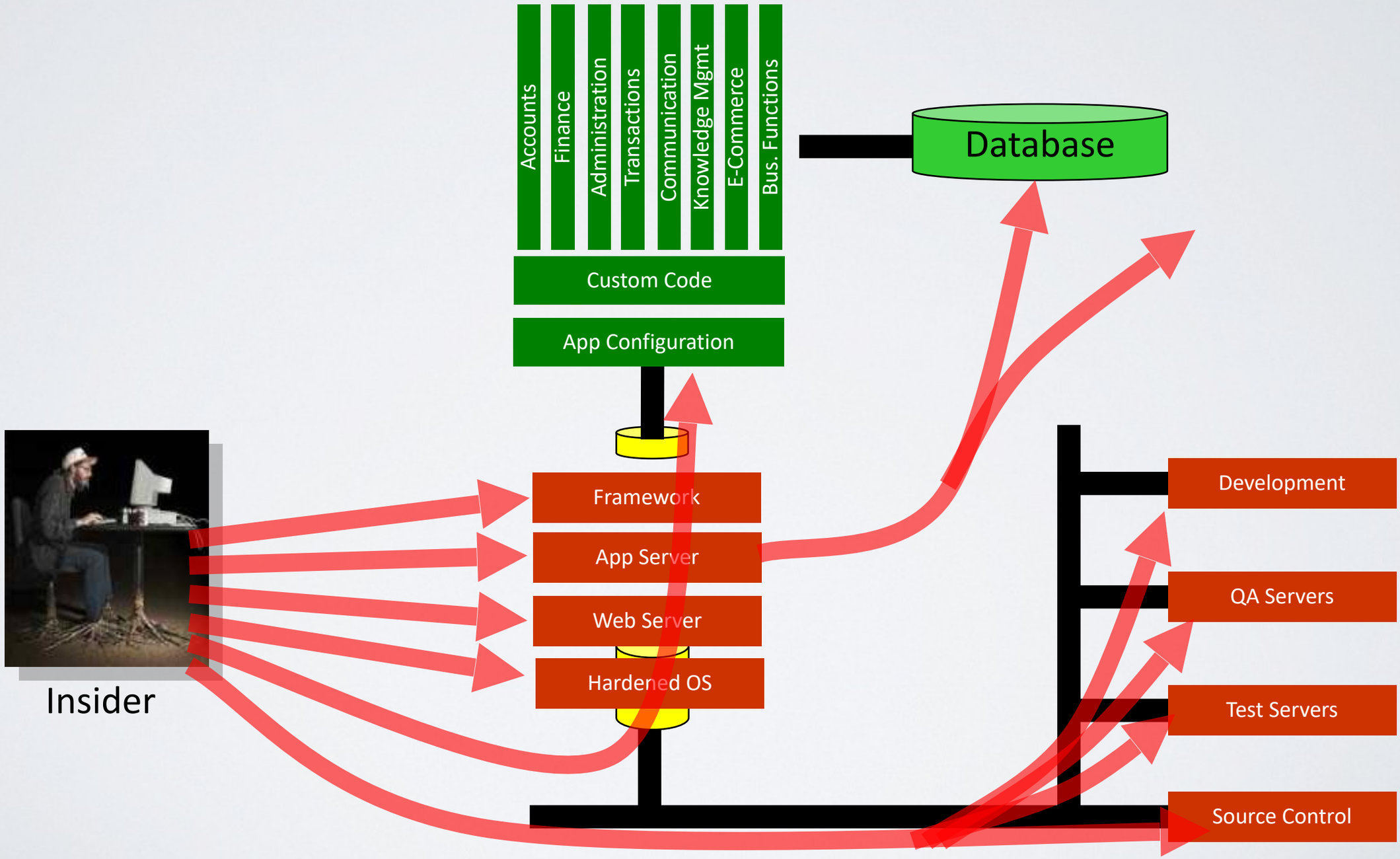
CM must extend to all parts of the application

- All credentials should change in production

Typical Impact

- Install backdoor through missing OS or server patch
- Unauthorized access to default accounts, application functionality or data, or unused but accessible functionality due to poor server configuration

SECURITY MISCONFIGURATION



A5 – SECURITY MISCONFIGURATION

Verify your system's configuration management

- Secure configuration "hardening" guideline
  - Automation is REALLY USEFUL here
- Must cover entire platform and application
- Analyze security effects of changes

Can you "dump" the application configuration?

- Build reporting into your process
- If you can't verify it, it isn't secure

Verify the implementation

- Scanning finds generic configuration and missing patch problems

A6 – SENSITIVE DATA EXPOSURE

Storing and transmitting sensitive data insecurely

- Failure to identify all sensitive data
- Failure to identify all the places that this sensitive data gets stored
  - Databases, files, directories, log files, backups, etc.
- Failure to identify all the places that this sensitive data is sent
  - On the web, to backend databases, to business partners, internal communications
- Failure to properly protect this data in every location

Typical Impact

- Attackers access or modify confidential or private information
  - e.g, credit cards, health care records, financial data (yours or your customers)
- Attackers extract secrets to use in additional attacks
- Company embarrassment, customer dissatisfaction, and loss of trust
- Expense of cleaning up the incident, such as forensics, sending apology letters, reissuing thousands of credit cards, providing identity theft insurance
- Business gets sued and/or fined

INSECURE CRYPTOGRAPHIC STORAGE

1 **Victim enters credit card number in form**

Accounts
Finance
Administration
Transactions
Communication
Knowledge Mgmt
E-Commerce
Bus. Functions

Custom Code

Log files

Error handler logs CC details because merchant gateway is unavailable 2

3 **Logs are accessible to all members of IT staff for debugging purposes**

4 **Malicious insider steals 4 million credit card numbers**

AVOIDING INSECURE CRYPTOGRAPHIC STORAGE

Verify your architecture

- Identify all sensitive data
- Identify all the places that data is stored
- Ensure threat model accounts for possible attacks
- Use encryption to counter the threats, don't just 'encrypt' the data

Protect with appropriate mechanisms

- File encryption, database encryption, data element encryption

AVOIDING INSECURE CRYPTOGRAPHIC STORAGE

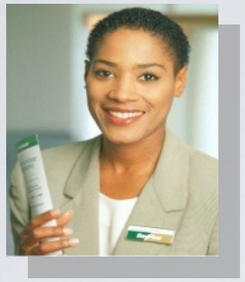
Use the mechanisms correctly

- Use standard strong algorithms
- Generate, distribute, and protect keys properly
- Be prepared for key change

Verify the implementation

- A standard strong algorithm is used, and it's the proper algorithm for this situation
- All keys, certificates, and passwords are properly stored and protected
- Safe key distribution and an effective plan for key change are in place
- Analyze encryption code for common flaws

INSUFFICIENT TRANSPORT LAYER PROTECTION

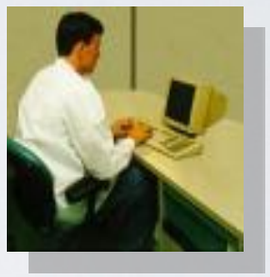


External Victim

Custom Code

Backend Systems

Business Partners



Employees

1

External attacker steals credentials and data off network

External Attacker

2

Internal attacker steals credentials and data from internal network

Internal Attacker

AVOIDING INSUFFICIENT TRANSPORT LAYER PROTECTION

Protect with appropriate mechanisms

- Use TLS on all connections with sensitive data
- Use HSTS (HTTP Strict Transport Security)
- Use key pinning
- Individually encrypt messages before transmission
  - E.g., XML-Encryption
- Sign messages before transmission
  - E.g., XML-Signature

Use the mechanisms correctly

- Use standard strong algorithms (disable old SSL algorithms)
- Manage keys/certificates properly
- Verify SSL certificates before using them
- Use proven mechanisms when sufficient

A7 – MISSING FUNCTION LEVEL ACCESS CONTROL

How do you protect access to URLs (pages)?

Or functions referenced by a URL plus parameters ?

- This is part of enforcing proper “authorization”, along with A4 – Insecure Direct Object References

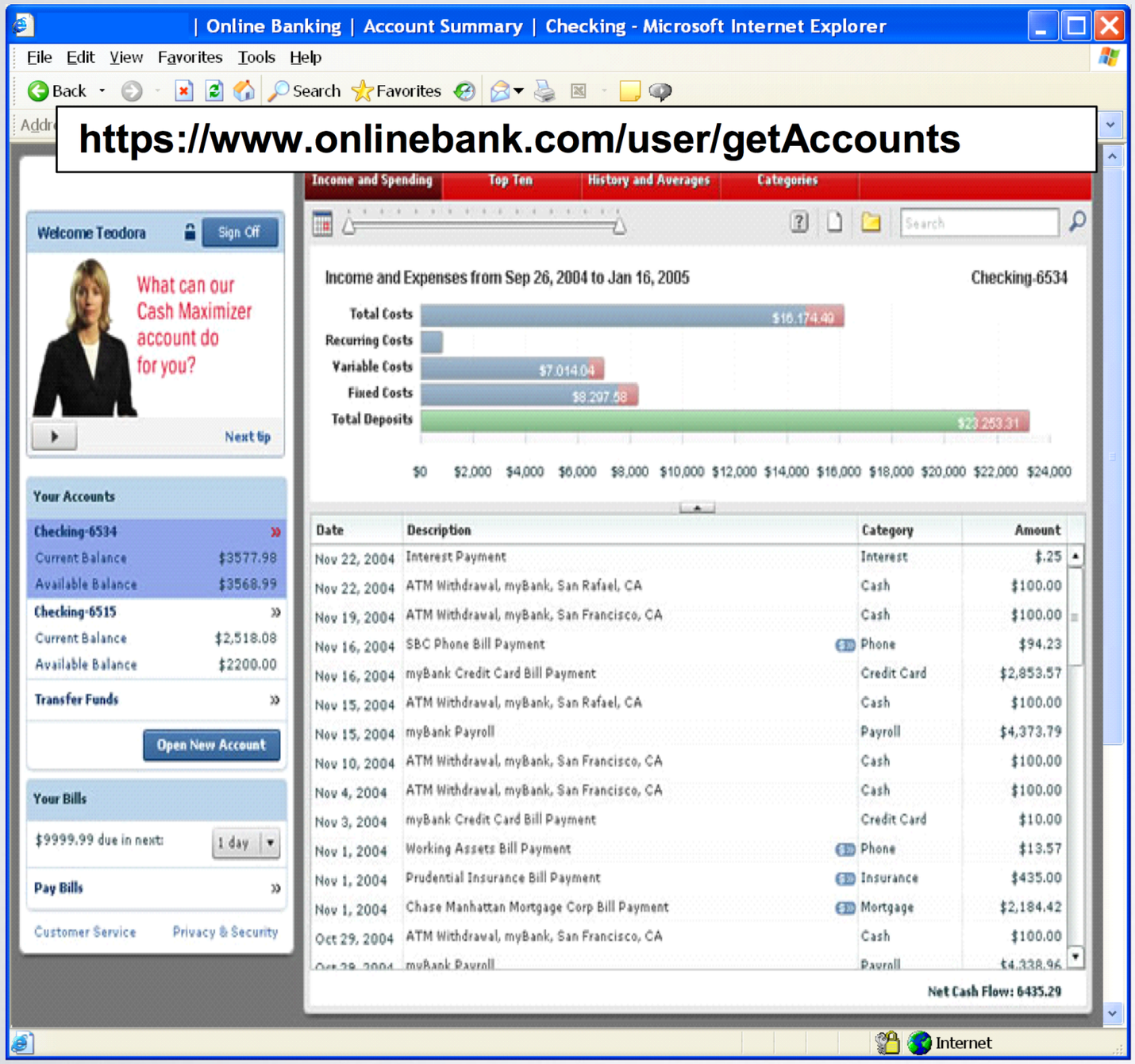
A common mistake …

- Displaying only authorized links and menu choices
- This is called presentation layer access control, and doesn’t work
- Attacker simply forges direct access to ‘unauthorized’ pages

Typical Impact

- Attackers invoke functions and services they’re not authorized for
- Access other user’s accounts and data
- Perform privileged actions

MISSING FUNCTION LEVEL ACCESS CONTROL ILLUSTRATED



- Attacker notices the URL indicates his role

  /user/getAccounts

- He modifies it to another directory (role)

  /admin/getAccounts, or

  /manager/getAccounts

- Attacker views more accounts than just their

AVOIDING MISSING FUNCTION LEVEL ACCESS CONTROL

For function, a site needs to do 3 things

- Restrict access to authenticated users (if not public)
- Enforce any user or role based permissions (if private)
- Completely disallow requests to unauthorized page types (e.g., config files, log files, source files, etc.)

Verify your architecture

- Use a simple, positive model at every layer
- Be sure you actually have a mechanism at every layer

Verify the implementation

- Forget automated analysis approaches
- Verify that each URL (plus any parameters) referencing a function is protected by
  - An external filter, like Java EE web.xml or a commercial product
  - Or internal checks in YOUR code – e.g., use ESAPI's isAuthorizedForURL() method
- Verify the server configuration disallows requests to unauthorized file types
- Use OWASP's ZAP or your browser to forge unauthorized requests

A8 – CROSS SITE REQUEST FORGERY (CSRF)

Cross Site Request Forgery

- An attack where the victim's browser is tricked into issuing a command to a vulnerable web application
- Vulnerability is caused by browsers automatically including user authentication data (session ID, IP address, Windows domain credentials, …) with each request

Imagine…

- What if a hacker could steer your mouse and get you to click on links in your online banking application?
- What could they make you do?

Typical Impact

- Initiate transactions (transfer funds, logout user, close account)
- Access sensitive data
- Change account details

CSRF VULNERABILITY PATTERN

The Problem

- Web browsers automatically include most credentials with each request
- Even for requests caused by a form, script, or image on another site

Conditions that must be met for CSRF attack

- **A relevant action** - an action on the vulnerable web site the attacker is interested in triggering.
- **Cookie-based session handling** - auth is based solely on cookie session ids or tokens
  - These are sent automatically by the browser!
- **No unpredictable request parameters** - the attacker knows everything that is needed to perform the request

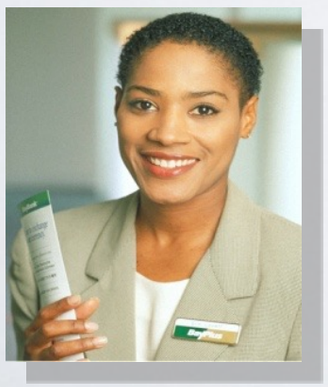
CSRF ILLUSTRATED

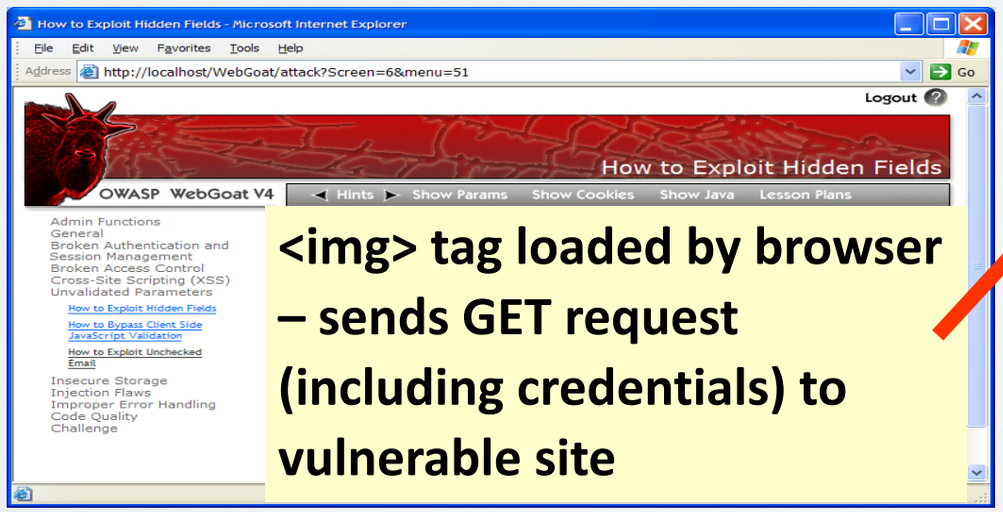
Attacker sets the trap on some website on the internet (or simply via an e-mail)

1

2

While logged into vulnerable site, victim views attacker site





Application with CSRF vulnerability

3

Vulnerable site sees legitimate request from victim and performs the action requested

A8 – AVOIDING CSRF FLAWS

Add a secret, not automatically submitted, token to ALL sensitive requests

- This makes it impossible for the attacker to spoof the request
  - (unless there's an XSS hole in your application)
- Tokens should be cryptographically strong or random

Options

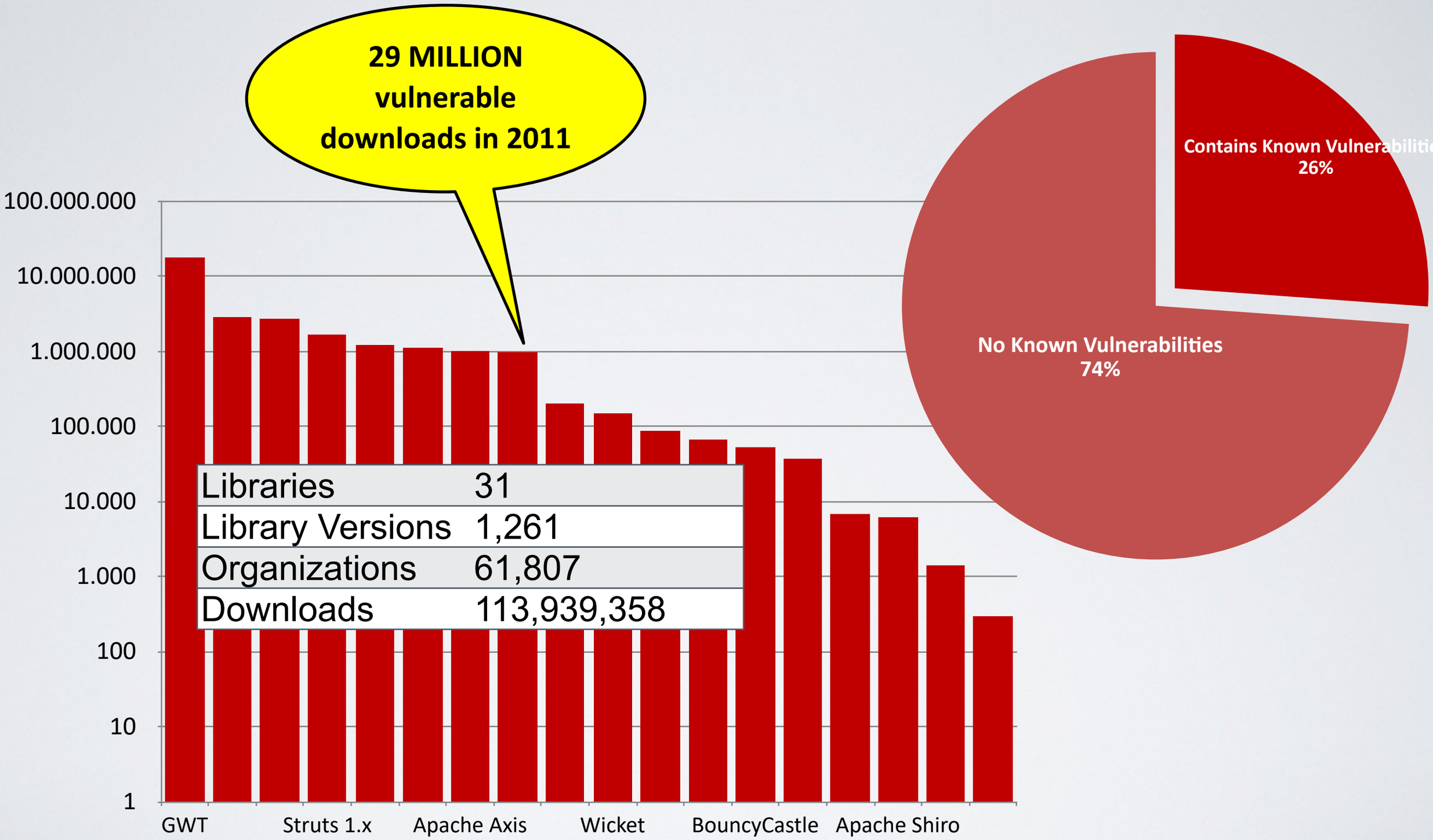
- Store a single token in the session and add it to all forms and links
  - Hidden Field: <input name="token" value="687965fdfaew87agrde" type="hidden"/>
  - Single use URL: /accounts/687965fdfaew87agrde
  - Form Token: /accounts?auth=687965fdfaew87agrde …
- Beware exposing the token in a referer header
  - Hidden fields are recommended
- Can have a unique token for each function
  - Use a hash of function name, session id, and a secret
- Can require secondary authentication for sensitive functions (e.g., eTrade)

Don't allow attackers to store attacks on your site

- Properly encode all input on the way out
- This renders all links/requests inert in most interpreters

See the: www.owasp.org/index.php/CSRF_Prevention_Cheat_Sheet for more details

EVERYONE USES VULNERABLE LIBRARIES



https://www.aspectsecurity.com/news/press/the-unfortunate-reality-of-insecure-libraries

A9 – USING KNOWN VULNERABLE COMPONENTS

Vulnerable Components Are Common

- Some vulnerable components (e.g., framework libraries) can be identified and exploited with automated tools
- This expands the threat agent pool beyond targeted attackers to include chaotic actors

Widespread

- Virtually every application has these issues because most development teams don't focus on ensuring their components/libraries are up to date
- In many cases, the developers don't even know all the components they are using, never mind their versions. Component dependencies make things even worse

Typical Impact

- Full range of weaknesses is possible, including injection, broken access control, XSS ...
- The impact could range from minimal to complete host takeover and data compromise

WHAT CAN YOU DO TO AVOID THIS?

Ideal

- Automation checks periodically (e.g., nightly build) to see if your libraries are out of date
- Even better, automation also tells you about known vulnerabilities

Minimum

- By hand, periodically check to see if your libraries are out of date and upgrade those that are
- If any are out of date, but you really don't want to upgrade, check to see if there are any known security issues with these out of data libraries
- If so, upgrade those

Could also

- By hand, periodically check to see if any of your libraries have any known vulnerabilities at this time
- Check CVE, other vulnerability repositories
- If any do, update at least these

AUTOMATION EXAMPLE FOR JAVA – USE MAVEN 'VERSIONS' PLUGIN

Output from the Maven Versions Plugin – Automated Analysis of Libraries' Status against Central repository

Dependencies

Status	Group Id	Artifact Id	Current Version	Scope	Classifier	Type	Next Version	Next Incremental	Next Minor	Next Major
⚠	com.fasterxml.jackson.core	jackson-annotations	2.0.4	compile		jar		**2.0.5**	**2.1.0**	
⚠	com.fasterxml.jackson.core	jackson-core	2.0.4	compile		jar		**2.0.5**	**2.1.0**	
⚠	com.fasterxml.jackson.core	jackson-databind	2.0.4	compile		jar		**2.0.5**	**2.1.0**	
⚠	com.google.guava	guava	11.0	compile		jar		**11.0.1**	**12.0-rc1**	**12.0**
⚠	com.ibm.icu	icu4j	49.1	compile		jar				**50.1**
⚠	com.theoryinpractise	halbuilder	1.0.4	compile		jar		**1.0.5**		
⚠	commons-codec	commons-codec	1.3	compile		jar			**1.4**	
✓	commons-logging	commons-logging	1.1.1	compile		jar				
⚠	joda-time	joda-time	2.0	compile		jar			**2.1**	
⚠	net.sf.ehcache	ehcache-core	2.5.1	compile		jar		**2.5.2**	**2.6.0**	
⚠	org.apache.httpcomponents	httpclient	4.1.2	compile		jar		**4.1.3**	**4.2**	
⚠	org.apache.httpcomponents	httpclient-cache	4.1.2	compile		jar		**4.1.3**	**4.2**	
⚠	org.apache.httpcomponents	httpcore	4.1.2	compile		jar		**4.1.3**	**4.2**	
⚠	org.jdom	jdom	1.1	compile		jar		**1.1.2**		**2.0.0**
✓	org.slf4j	slf4j-api	1.7.2	provided		jar				

Most out of Date!

Details Developer Needs

This can automatically be run EVERY TIME software is built!!

A10 – UNVALIDATED REDIRECTS AND FORWARDS

Web application redirects are very common

- And frequently include user supplied parameters in the destination URL
- If they aren't validated, attacker can send victim to a site of their choice

Forwards (aka Transfer in .NET) are common too

- They internally send the request to a new page in the same application
- Sometimes parameters define the target page
- If not validated, attacker may be able to use unvalidated forward to bypass authentication or authorization checks

Typical Impact

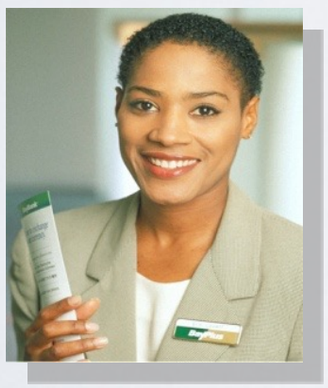
- Redirect victim to phishing or malware site
- Attacker's request is forwarded past security checks, allowing unauthorized function or data access

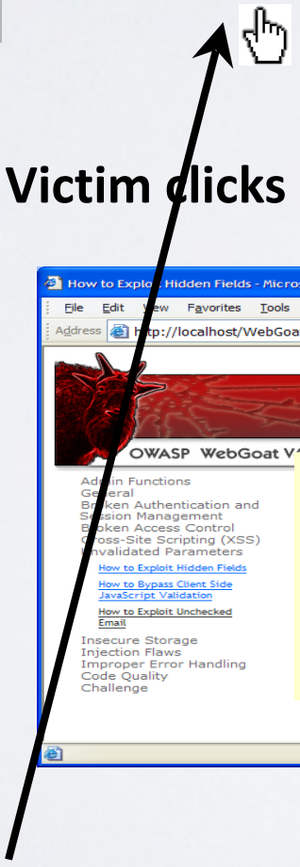
UNVALIDATED REDIRECT ILLUSTRATED

1 **Attacker sends attack to victim via email or webpage**

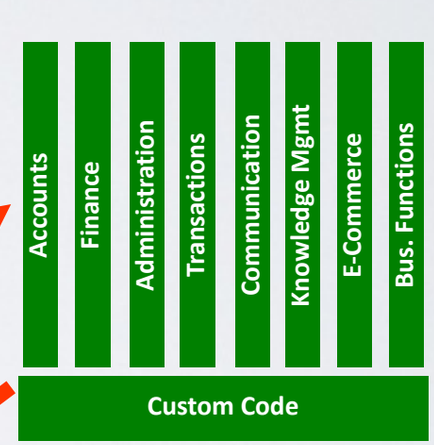
From: Internal Revenue Service
Subject: Your Unclaimed Tax Refund
Our records show you have an unclaimed federal tax refund. Please click here to initiate your claim.

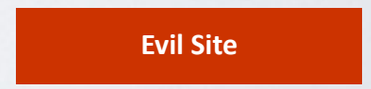
2 **Victim clicks link containing unvalidated parameter**





3 **Application redirects victim to attacker's site**



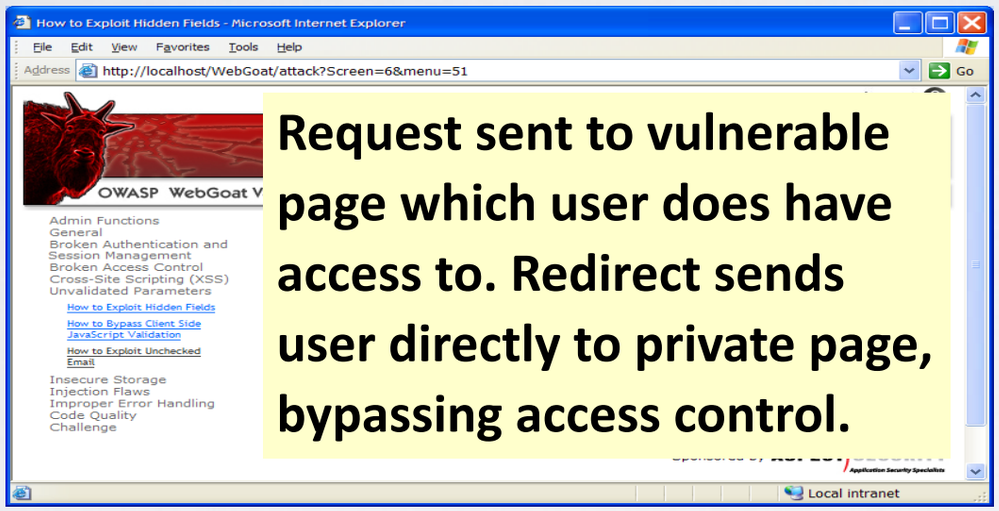


4 **Evil site installs malware on victim, or phish's for private information**

http://www.irs.gov/taxrefund/claim.jsp?year=2006& ...
&dest=www.evilsite.com

UNVALIDATED FORWARD ILLUSTRATED

1 **Attacker sends attack to vulnerable page they have access to**




```
public void
sensitiveMethod( HttpServletRequest
request, HttpServletResponse response) {
    try {
            // Do sensitive stuff here.
            ...
  }
  catch ( ...
```

2 **Application authorizes request, which continues to vulnerable page**

Filter

```
public void doPost( HttpServletRequest request,
HttpServletResponse response) {
    try {
        String target = request.getParameter( "dest" ) );
    ...
    request.getRequestDispatcher( target ).forward(request,
      response);
  }
  catch ( ...
```

3 **Forwarding page fails to validate parameter, sending attacker to unauthorized page, bypassing access control**

A10 – AVOIDING UNVALIDATED REDIRECTS AND FORWARDS

There are a number of options

- Avoid using redirects and forwards as much as you can
- If used, don't involve user parameters in defining the target URL
- If you 'must' involve user parameters, then either
  - Validate each parameter to ensure its valid and authorized for the current user, or
  - (preferred) – Use server side mapping to translate choice provided to user with actual target page
- Defense in depth: For redirects, validate the target URL after it is calculated to make sure it goes to an authorized external site
- ESAPI can do this for you!!

A10 – AVOIDING UNVALIDATED REDIRECTS AND FORWARDS

Some thoughts about protecting Forwards

- Ideally, you'd call the access controller to make sure the user is authorized before you perform the forward (with ESAPI, this is easy)
- With an external filter, like Siteminder, this is not very practical
- Next best is to make sure that users who can access the original page are ALL authorized to access the target page.

SUMMARY: HOW DO YOU ADDRESS THESE PROBLEMS?

Develop Secure Code

- Follow the best practices in OWASP's Guide to Building Secure Web Applications
  - https://www.owasp.org/index.php/Guide
  - And the cheat sheets: https://www.owasp.org/index.php/Cheat_Sheets
- Use OWASP's Application Security Verification Standard as a guide to what an application needs to be secure
  - https://www.owasp.org/index.php/ASVS
- Use standard security components that are a fit for your organization
  - Use OWASP's ESAPI as a basis for your standard components
  - https://www.owasp.org/index.php/ESAPI

Review Your Applications

- Have an expert team review your applications
- Review your applications yourselves following OWASP Guidelines
  - OWASP Code Review Guide: https://www.owasp.org/index.php/Code_Review_Guide
  - OWASP Testing Guide: https://www.owasp.org/index.php/Testing_Guide